FastGraph: PCA-Subspace Binning for Exact GPU Neighbor Search in Calorimeter Reconstruction

Aarush Agarwal∗ aarusha@andrew.cmu.edu
Carnegie Mellon University
Pittsburgh, PA, USA

Jan Kieseler jan.kieseler@cern.ch
Karlsruhe Institute of Technology
Karlsruhe, Germany

Raymond He rhe2@andrew.cmu.edu
Carnegie Mellon University
Pittsburgh, PA, USA

Matteo Cremonesi mcremone@andrew.cmu.edu
Carnegie Mellon University
Pittsburgh, PA, USA

Shah Rukh Qasim shah.rukh.qasim@cern.ch
University of Zurich
Zürich, Switzerland

Abstract

Separating overlapping particle showers in the CMS High Granularity Calorimeter (HGCAL) requires reconstruction methods that exploit sparse, irregular energy deposits. GravNet-based approaches exchange information between hits through neighbor graphs in learned coordinate spaces, requiring repeated graph construction. We present **FastGraph**, a CUDA/PyTorch implementation of exact k-nearest-neighbor search motivated by this computational task. It uses a principal component analysis (PCA) subspace for cell-list pruning while evaluating candidate distances in the original feature space. An orthonormal projection supplies a lower bound for exact search in exact arithmetic; finite-precision correctness is assessed separately in bounded tests. The study covers d=2–10, with PCA used at d=4–10, and includes an opt-in differentiable GravNet integration. On a 5 M-point, single-segment HGCAL detector-feature stress test at d=8, k=40, FastGraph constructs the graph in 7.5 s, with speedups of up to 41× over FAISS-GPU and 19× over cuVS brute force. The measured speedup over approximate CAGRA is 17×, without a matched-recall comparison. These results establish a kernel-level advantage on the tested detector-feature distributions; event-batched learned-coordinate workloads, reconstruction performance, and end-to-end model throughput remain to be evaluated.

Keywords: Calorimeter Reconstruction, Computational Particle Physics, k-Nearest Neighbors, GPU Computing, Principal Component Analysis, Graph Neural Networks

1. Introduction

Calorimetric particle reconstruction begins with many local energy deposits and must infer the particles that produced them. An incident electron, photon, or hadron can initiate

∗Corresponding author

a shower of secondary particles; nearby showers can overlap, making the assignment of measured deposits to individual particles ambiguous. This challenge is particularly acute in the High-Luminosity Large Hadron Collider, where additional proton–proton collisions in the same bunch crossing (pileup) increase the density of energy deposits. The CMS High Granularity Calorimeter (HGCAL) is designed to resolve shower structure through fine transverse and longitudinal segmentation [1]. Exploiting that information requires algorithms that separate showers while retaining the information needed to estimate particle energies and positions. Published HGCAL studies have therefore explored joint hit clustering, particle identification, and energy regression within computational and memory constraints [2].

The computational representation matters because detector hits form a sparse, irregular point set rather than a regular image. A graph neural network (GNN) represents each hit as a node and uses edges to exchange information between selected hits. In GravNet [3], the network maps hit features to learned coordinates and connects each hit to a fixed number of nearby points in that coordinate space. Distance-weighted aggregation then updates the hit representation. These learned coordinates express similarity relevant to reconstruction; they need not coincide with physical detector positions, and a neighbor edge is not itself an assignment to a shower. Together with an object condensation loss, which trains the network to organize hits into reconstructable objects, this approach has been investigated for multi-particle calorimeter reconstruction [4, 2, 5].

Constructing these neighborhoods is a recurring numerical task. For N hits with d coordinates, exhaustive all-query neighbor search requires $O(N^2 d)$ distance work. Unlike a static detector adjacency, a learned-coordinate graph must be rebuilt when the coordinates change between network layers or forward passes. The original GravNet study uses d=4 for this search [3], motivating the d=4–10 range examined here. Reducing graph-construction cost could make such reconstruction models less expensive to execute; the gain in a complete model also depends on feature transformations, message passing, and backpropagation. We therefore study neighbor construction as a computational component and do not assume that its speedup transfers unchanged to event reconstruction.

A useful starting point for computational physics is the cell list, familiar from molecular-simulation neighbor searches [6, 7]. Points are assigned to spatial bins so that nearby bins can be searched without evaluating every pairwise distance. A grid with n_{bins} divisions along each of d axes contains n_{bins}^d cells, however, making a full grid costly even at moderate d. FastGraph instead bins in a small principal-component subspace and evaluates candidate distances using all original coordinates. The projection serves only as a search index: it does not replace the distance metric chosen by the reconstruction model. Because an orthonormal projection cannot increase a distance, bounds in the binning space can certify termination of the full-space search in exact arithmetic. Section 3.4 states the assumptions and distinguishes this argument from FP32 validation.

The contribution is the formulation and CUDA/PyTorch realization of this sampled-PCA cell-list search for repeated, low-to-moderate-dimensional graph construction. PCA indexing and spatial pruning are established ideas; our study examines their combination with full-space distance evaluation, event-local search support, and distance gradients in a GPU graph-construction primitive. Exact search provides a specified distance criterion without an approximate index's recall parameter, although tied distances can admit several valid edge sets. We do not establish that exact neighbors improve HGCAL physics performance over approximate neighbors.

We evaluate the implementation on detector-derived HGCAL hit features and on con-

trolled synthetic distributions. The HGCAL benchmark concatenates events into one segment and uses stored feature columns, whose semantics and preprocessing are unavailable in the supplied artifact; it is a kernel stress test rather than a sample of learned coordinates from a reconstruction model. At d=8, N=5 M, k=40, FastGraph takes 7.5 s versus 310 s for exact FAISS-GPU and 146 s for exact cuVS brute force. Across d=2–10 at this (N, k) point, geometric-mean speedups are 10.4× and 5.0×, respectively. The d=2–3 measurements are axis-aligned controls; PCA is used at d=4–10. The synthetic study and the direct three-dimensional comparison identify distributions on which the advantage decreases or other methods win. These tests establish where the computational component is effective; event-preserving learned-coordinate timings and reconstruction observables remain necessary to assess its benefit to the motivating physics application.

The following sections place the method among existing neighbor-search approaches, derive its search criterion, and describe its implementation. The performance study then separates detector-feature timings, distribution and parameter controls, memory measurements, and correctness checks before discussing the limits of the evidence.

2. Related Work

Neighbor search in a reconstruction model can be accelerated either by evaluating all distances efficiently or by reducing the candidate set. The first approach maps naturally to GPUs. FAISS [8] provides a highly tuned `IndexFlatL2` exact brute-force GPU index together with the fast k-selection primitives that downstream methods reuse. RAPIDS `cuVS` exposes a comparable exact brute-force GPU implementation that we use as the strongest exact baseline in this work; KeOps [9] provides symbolic matrix reductions with automatic differentiation that can express exact kNN without materializing the full distance matrix. Direct exact GPU kNN-graph construction has also been studied in the multi-GPU and cluster settings [10, 11]. PyTorch Geometric exposes the widely used `knn_graph` interface through its `torch_cluster` extension [12]; we treat it as a framework interface rather than a distinct algorithmic baseline. FastGraph is not the first exact GPU kNN library; it specializes the accelerator for the small-d exact graph-construction setting that arises in dynamic GNN layers.

Reducing candidate work instead requires a spatial index. Cell lists are widely used in molecular simulations, including GPU neighbor-list construction [6, 7]. For nearest-neighbor search, points can be sorted by bin and searched through expanding shells until the current neighbor radius excludes all unvisited cells. Their grid storage grows exponentially with the number of binning axes, which motivates indexing only a few coordinates. Qasim's PhD thesis [13] discusses the practical grid dimensionality limit in the GNN context. FastGraph uses a principal subspace for the grid coordinates; an axis-aligned first-column choice is retained only as an internal ablation.

Choosing those coordinates adaptively connects cell lists to a broader history of projected indexing. Using a principal-component projection as a coordinate system for spatial indexing has a long history; Sproull's k-d tree refinements [14] already noted that splitting on the direction of maximum variance yields more balanced partitions than splitting on a coordinate axis. Principal-axis tree variants use a similar idea for CPU-side neighbor search. Neither PCA indexing nor lower-bound pruning is new by itself. FastGraph applies a partial top-k projection to a uniform GPU cell list, retains original d-dimensional distances, and exposes the resulting search to dynamic graph layers. The contraction $V_k V_k^\top \preceq I_d$ supplies its

exact-arithmetic termination criterion (Section 3); the experiments assess the benefit of this choice of binning basis.

Other spatial indices provide particularly strong alternatives in three dimensions. A separate line of work targets exact GPU kNN through dedicated spatial-acceleration structures that exploit the GPU's ray-tracing hardware or BVH machinery. RTNN [15], RT-kNNS Unbound [16], and Arkade [17] retarget the OptiX BVH unit for neighbor search and report large speedups over shader-core baselines in 3D. LBVH-kNN [18] (`cupy-knn`) attains similar gains via a left-balanced LBVH built without dedicated RT hardware. CLOVER [19] introduces a GPU-native, Voronoi spatio-graph approach to exact kNN and reports 4× over an "optimized grid" baseline and 230× over FAISS on low-dimensional spatial data. All of these methods are implemented and evaluated for two- or three-dimensional inputs: Arkade notes that RT cores build BVHs only on three-dimensional data, CLOVER's reference implementation asserts `D == 3`, and LBVH-kNN is evaluated exclusively in 3D. We include a direct head-to-head against CLOVER at d=3 in Section 5.6, but the released methods do not provide a d=4–10 path comparable to the one evaluated here.

When exact neighbors are unnecessary, GPU-resident approximate nearest neighbor (ANN) systems offer a further option. SONG [20] decomposes graph traversal into GPU-friendly stages; GGNN [21] builds a hierarchical kNN graph for index construction and search; CAGRA [22], distributed as part of RAPIDS `cuVS`, builds a high-quality graph index with an IVF-PQ or NN-Descent initialization. ANN-Descent variants have also been adapted to reduce memory traffic during construction [23]. These systems expose a tunable recall–throughput tradeoff for high-throughput vector retrieval; FastGraph instead targets exact, small-d graph construction. We benchmark CAGRA and GGNN as approximate comparators (Section 5.7) but do not include CPU-only ANN systems (HNSW, ScaNN, Annoy) [24, 25, 26] in the timing tables: the point tensors and large intermediate arrays remain GPU-resident, and a CPU-side detour for bulk data would not be a fair comparator.

The relevance of these search methods to physics depends on how the graph enters the reconstruction algorithm. Beyond GravNet and object condensation, dynamic neighbor graphs also appear in particle classification and tracking. DGCNN/EdgeConv recomputes the graph in feature space at each layer [27]; ParticleNet adapts the same idea to jet tagging [28]; GravNet/GarNet layers [3] and the object condensation loss [4] have been studied for HGCAL reconstruction [2, 5]; the HEP.TrkX/Exa.TrkX line of work uses learned-embedding graphs for tracking [29, 30, 31]. FastGraph supplies a computational primitive for the neighbor-search step in such models. This study concentrates on the low-dimensional search regime motivated by calorimetry; it does not evaluate the wider feature spaces that can occur in later EdgeConv or ParticleNet layers.

3. The PCA-Subspace Binned kNN Algorithm

In the algorithm and complexity analysis, K denotes the requested neighborhood size (including self), and k denotes the binning-subspace dimension. Experimental figure labels use the conventional k for neighborhood size; binning dimensions are identified explicitly.

3.1. Phase 1: Subspace estimation

Given the input coordinates $X \in \mathbb{R}^{N\times d}$ for the current call, FastGraph samples up to 50,000 rows, centers that sample, and computes the top-k principal axes with `torch.pca_lowrank`. The current release fits one projection for the complete coordinate tensor rather than a separate projection for each `row_splits` segment. The number of

binning axes k is a user-facing hyperparameter; we default to $k = 3$ for the HGCAL workloads in this paper, justified empirically in the ablation of Section 5.5. The resulting projection matrix $V_k \in \mathbb{R}^{d\times k}$ satisfies $V_k^\top V_k = I_k$ and $V_k V_k^\top \preceq I_d$. The projected coordinates used for binning are $Y = XV_k$. PCA estimation is performed once per call on the GPU and is included in the reported FastGraph timing. The random subsample follows PyTorch's active random number-generator state.

This construction simplifies when the input dimensionality d does not exceed the binning dimensionality k, FastGraph skips PCA estimation entirely and binds the binning axes to the raw input axes; the projection V_k reduces to the identity (up to permutation), $Y = X$, and the algorithm behaves exactly as a classical axis-aligned cell list. With the default $k = 3$, the $d \in \{2, 3\}$ gains therefore come from the underlying cell-list machinery; the PCA contribution begins at $d > k$, covering the d=4–10 detector-feature experiments motivated by the GravNet search dimension.

3.2. Phase 2: Binning in the projected subspace

We partition Y with a uniform grid whose scalar width w is shared across the k projected axes. The division count is set heuristically to balance bin density against neighborhood reach,

$$n_{\text{bins}} = \left(\frac{32\, n_{\text{elems}}}{K}\right)^{\frac{1}{k}},$$

where the released implementation uses the historical occupancy proxy

$$n_{\text{elems}} = \lfloor \max(\texttt{row_splits})/|\texttt{row_splits}| \rfloor + 1,$$

and K is the requested neighborhood size. The integer division count is clamped to $[5, 30]$. For the one-segment benchmark input `row_splits`=$[0, N]$, this proxy is approximately $N/2$; it is a performance heuristic and does not enter the exactness condition. Points are sorted by bin index, and cumulative offsets are stored so that each bin can be addressed as a contiguous slice of the sorted point array. Row splits, which delineate the per-event boundaries inside a batched tensor, are honored by the binning kernel so that no query bin spans events.

3.3. Phase 3: Per-query search with full-dimensional distances

For each query x_q, the search starts in the bin containing $y_q = x_q V_k$ and expands through successive shells of k-D hypercube bins. For every candidate u in a visited bin the *full-dimensional* squared Euclidean distance $\|x_q - x_u\|^2 = \sum_{i=1}^{d}(x_{q,i} - x_{u,i})^2$ is computed (not the k-dimensional projected distance). The current best neighbor radius r_K and a fixed-size top-K candidate buffer are maintained incrementally.

3.4. Why this is exact: the contraction argument

For finite inputs with valid row splits, a supported projection dimension $2 \le k \le d$, and at least K eligible points in each segment (or in its direction-masked subset), consider the standard cube-radius termination test applied to bins in the projected subspace while all candidate distances are evaluated in the full input space. The exactness of this test rests on a single observation about the partial projection. For any two points $x_a, x_b \in \mathbb{R}^d$,

$$\|y_a - y_b\|^2 = (x_a - x_b)^\top V_k V_k^\top (x_a - x_b) \le \|x_a - x_b\|^2,$$

because $V_k V_k^\top$ has eigenvalues in $\{0, 1\}$ and is therefore $\preceq I_d$. Equivalently, the projection is contractive: the projected distance never overestimates the true distance.

After visiting all bins within the current hypercube shell of radius s, any unvisited bin lies at projected distance at least $w \cdot s$ from y_q, where w is the scalar width shared by all projected axes in the released implementation. Thus any unvisited point satisfies $\|y_q - y_u\|^2 \geq (w \cdot s)^2$. Combined with the contraction above, $\|x_q - x_u\|^2 \geq (w \cdot s)^2$, so once the top-$K$ buffer has filled and $(w \cdot s)^2 > r_K^2$, no unseen point can be closer in the true distance than the current K-th neighbor. The algorithm may safely terminate; the returned K neighbors are exact in exact arithmetic. This argument assumes an orthonormal projection and does not bound floating-point errors in the projection or pruning test. The FP32 implementation is validated separately against brute force at tractable scale, with the distance criteria and numerical tolerances reported in Section 5.7.

3.5. Search implementation

The `binstepper` is the small bookkeeping utility that enumerates the surface of the s-th hypercube shell around the query bin. It linearises each shell into a flat sequence, converts each step back into per-dimension offsets, discards interior cells, and yields the next surface bin's flat index. Exhaustive enumeration of the required shell cells is necessary for the termination argument; the utility separates this bookkeeping from candidate-distance evaluation in Algorithm 1. Keeping the shell enumerator separate from the distance kernel also makes the projected walk event-local: the same row-split offsets and contiguous bin slices can be reused across all queries in the batch.

Algorithm 1 makes the projected-vs-original-space distinction explicit: the bin indices (`binIdx`) and the stepper operate on y-space; the distance function `calcDist` operates on the original x-space coordinates; the termination test references the projected-space cube radius; the contraction argument certifies the result. This separation keeps the implementation and the proof aligned: projected-space search only decides which bins to visit, while exactness is enforced by the original-space distance computation.

3.6. Complexity

For a single segment, let $B = n_{\text{bins}}^k$ be the size of the bin grid; for multiple segments, let B denote their total allocated bin count. Both PCA-subspace binning and the axis-aligned k-coordinate control avoid a full n_{bins}^d grid. PCA changes the choice of binning axes rather than the grid's dimensional dependence.

To quantify setup work, let $S = \min(N, 50{,}000)$ be the PCA sample size and $q = \min(k+2, d)$ the low-rank solve size. For the fixed number of randomized iterations used by `torch.pca_lowrank`, the subspace fit costs $O(Sdq + (S+d)q^2)$, followed by $O(Ndk)$ to project all points. This is a sampled low-rank computation, not a full $O(Nd^2 + d^3)$ eigendecomposition. Binning, sorting, and offset construction take $O(N \log N + B)$.

The associated auxiliary memory is $O(N(d+k+K) + B)$: sorted coordinates, projected coordinates, permutation, neighbor indices, and bin offsets.

Search work depends on the input distribution. Let C_v be the number of candidate points evaluated for query v, and E_v the number of cell-offset tests performed by the shell enumerator, including rejected offsets. Each candidate requires $O(d)$ distance work and at most $O(K)$ for the top-K update; cell-offset bookkeeping costs $O(k)$ per test. The query cost is therefore $O(kE_v + C_v(d+K))$. Uniform projected density alone does not imply $C_v = O(K)$: many points may be close in projection but far apart in the discarded dimensions. When pruning and bin occupancy keep both C_v and E_v proportional to K, this reduces to $O(K(d+K))$ for fixed k. In the worst case, every point in a segment is evaluated, giving $O(N_v(d+K))$ candidate work for segment size N_v, in addition to shell enumeration.

Algorithm 1: PCA-Subspace Binned-Select-kNN: bins live in projected y-space; distances are computed in original x-space. Slot 0 is the query point itself, matching the FastGraph tensor convention; downstream message-passing code may drop or ignore this self edge.

Input: `coords`$[n_v][n_c]$ (original-space X), V_k, `binIdx`$[n_v]$ (projected-space bins), `binOffsets`$[n_v][n_b{+}1]$, `binBounds`$[n_B]$, `binCounts`$[n_b]$, scalar `binWidth`, n_v, K, n_c, n_b, n_B

Output: `neighbors`$[n_v][K]$ indices including slot 0 self by convention, r_K^2 values

for each $v \in [0, n_v)$ ***in parallel*** **do**
- `neighbors`[v][0]$\leftarrow (v, 0)$; `filled`$\leftarrow 1$; (`maxIdx`, `maxD2`) $\leftarrow (0, 0)$
- `totalBins` $\leftarrow \prod_{d=0}^{n_b-1}$ `binCounts`$[d]$, `gOff` $\leftarrow$ `totalBins` $\cdot \lfloor$`binIdx`$[v]$/`totalBins`$\rfloor$
- `step`$\leftarrow$`BinStepper`(`binCounts`, `binOffsets`[v]$[1..n_b]$) `// operates in projected` y`-space`
- $w \leftarrow$ `binWidth`
- `cont`$\leftarrow$ true; $s \leftarrow 0$ `// s: current cube-shell radius (projected space)`
- **while** *cont* **do**
  - `step.setDistance`(s); `cont`$\leftarrow$ false
  - **while** (`off` $\leftarrow$ `step.step`$()$) $\neq -1$ **do**
    - `idx` $\leftarrow$ `off` + `gOff`
    - **if** *idx* < 0 ***or*** *idx* $\geq n_B - 1$ **then**
      - **continue**
    - $(s_b, e_b) \leftarrow$ (`binBounds`[idx], `binBounds`[idx+1])
    - **if** $s_b = e_b$ **then**
      - `cont`$\leftarrow$ true; **continue**
    - **for** $u \in [s_b, e_b)$ **do**
      - **if** $u{=}v$ **then**
        - **continue**
      - `d2` $\leftarrow$ `calcDist`(v, u) `// full` d`-dim` $\sum_i (x_{v,i} - x_{u,i})^2$`, NOT projected`
      - **if** *filled* $< K$ **then**
        - `neighbors`[v][`filled`]$\leftarrow (u,$ `d2`$)$; **if** *d2* $>$ *maxD2* **then**
          - (`maxIdx`, `maxD2`) $\leftarrow$ (`filled`, `d2`)
        - ; `filled`++
      - **else if** *d2* $<$ *maxD2* **then**
        - `neighbors`[v][`maxIdx`]$\leftarrow (u,$ `d2`$)$;
        - (`maxIdx`, `maxD2`) $\leftarrow$ `findMaxDist`(`neighbors`$[v]$)
    - `cont`$\leftarrow$ true
  - **if** *filled*$=K$ ***and*** $(w \cdot s)^2 >$ *maxD2* **then**
    - **break**
  - s++

Combining setup with the work summed across queries gives

$$O\left(Sdq + (S+d)q^2 + Ndk + N\log N + B + \sum_{v=1}^{N}\left[kE_v + C_v(d+K)\right]\right).$$

Queries execute in parallel on the GPU; elapsed time also depends on load imbalance, memory traffic, and resource occupancy.

3.7. Gradient flow and PyTorch integration

FastGraph exposes gradients through the continuous outputs of the kernel—the selected squared distances and the input coordinates— while treating the discrete neighbor indices and the projection matrix V_k as constants in the backward pass. Concretely, the forward returns $(\text{idx}, \text{dist}^2)$ and the backward applies the chain rule with respect to dist^2 and the input coordinates, yielding a subgradient of the piecewise-smooth loss induced by the hard neighbor selection. This is the same gradient model used by any fixed-index distance layer (e.g., `torch.cdist` followed by `topk`) and integrates naturally into existing GNN autograd graphs. Treating V_k as fixed per forward–backward pair keeps graph selection consistent within that pair; the projection is refreshed at the next forward. The CUDA backward uses atomic accumulation and is therefore not guaranteed to be bitwise deterministic.

The PCA entry point is an eager-mode Python wrapper around the scripted axis-aligned primitive and the custom CUDA operation. A minimal usage example is:

```
from fastgraphcompute import binned_select_knn_pca
from fastgraphcompute.gnn_ops import GravNetOp

# Direct call (eager mode)
idx, dist2 = binned_select_knn_pca(K=40, coords=x, row_splits=rs,
                                   max_bin_dims=3)
# idx:   (N, K) int64 neighbor indices (constant in backward)
# dist2: (N, K) float32 squared distances (differentiable)
loss = model(dist2).sum()
loss.backward()  # gradients flow through dist2 to x

# Opt-in PCA neighbor construction inside GravNetOp (eager mode)
layer = GravNetOp(in_channels=16, out_channels=32,
                  space_dimensions=8, propagate_dimensions=32,
                  k=40, use_pca=True, max_bin_dims=3)
out, idx, dist2, space = layer(features, rs)
```

Listing 1: Minimal eager-mode PCA-FastGraph usage.

4. Experimental Setup

The experiments ask whether the proposed search reduces graph-construction cost on detector-derived inputs, how that behavior depends on the input distribution and binning dimension, and what correctness evidence supports the implementation. All GPU experiments use a single NVIDIA A100 80 GB PCIe with CUDA 12.1 and PyTorch 2.5. Compared backends are exact FAISS-GPU `IndexFlatL2` [8], cuVS brute force (also exact), CAGRA

via cuVS [22] (approximate), and GGNN [21] (approximate); FastGraph is compiled from the released source identified in the availability statement.

The detector-derived benchmark matrix comprises 1.16 billion CMS HGCAL simulation recHits concatenated across events, with ten stored feature columns per hit. A recHit is a reconstructed calorimeter energy deposit; the benchmark uses the stored numerical vectors without inferring their physical column definitions. The harness deterministically takes the first N rows and first d columns; the available artifact does not identify the column names, units, preprocessing, or normalization. Increasing d therefore changes both the number and identity of included features, and PCA remains sensitive to their numerical scales. Each timed slice is passed to the implementations as one segment rather than preserving event boundaries. This construction is a large-N stress test of the kernels on detector-derived features; it is not an event-batched GNN workload or a learned-coordinate sample. Synthetic comparisons use isotropic Gaussian $\mathcal{N}(0, I_d)$ samples, allowing dimensionality to be varied without the changing feature identities and scales of the detector-derived matrix.

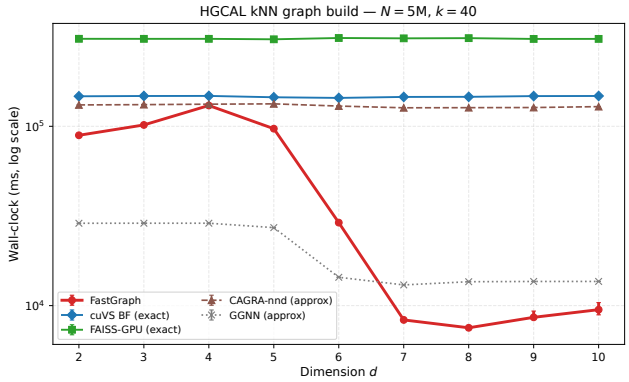
To compare computational cost on these inputs, all HGCAL timing plots and the headline table use the completed matched-input, serialized `-gres=mps:100` campaign only; older `mps:50` and exclusive-allocation rows are excluded. Every backend has the same input protocol. FastGraph/PCA, FAISS, cuVS BF, and GGNN have seven repetitions per completed plotted cell; CAGRA-NN-Descent has three. Inputs are copied to a Torch CUDA tensor before timing: FastGraph and GGNN consume that tensor directly, FAISS uses its Torch bridge, and cuVS BF and CAGRA use a pointer-identical CuPy DLPack view created before the timer. The timed region therefore includes index construction and search, but excludes data loading and host-to-device transfer for every backend. All timed regions are bracketed by device synchronization. Figures and scalar comparisons report medians across successful repetitions; error bars show the interquartile range.

One backend requires a different index-construction path for these inputs. The default CAGRA IVF-PQ build path fails in the 5 M-point timing campaign at $d \geq 5$ with a RAFT assertion about invalid or duplicated neighbor nodes. We did not isolate the internal cause of this failure. We therefore use CAGRA's `build_algo="nn_descent"` path for the headline timing campaign; it avoids the PQ initialization, succeeds across the full d=2–10 sweep, and is the robust full-sweep CAGRA timing baseline. The archived recall diagnostic predates that campaign and is labeled separately as default-build CAGRA.

For FastGraph, the binning subspace size k is exposed as `max_bin_dims`. The benchmark binary and evaluated kernel surface provide compile-time template instantiations for $k \in \{2, 3, 4, 5\}$; we use $k = 3$ as the default for HGCAL and ablate that surface in Section 5.5. The immutable source release also contains experimental $k = 6, 7$ dispatches used to probe the design boundary, but they are not part of the recommended headline configuration; $k = 7$ exceeds the current per-block register budget for some configurations. The empirical optimum on HGCAL data is well inside the evaluated range.

5. Performance Comparisons

We first compare construction times on the detector-feature stress test, then use synthetic inputs and a binning ablation to examine the conditions behind those results. Memory and recall measurements have distinct protocols, stated below, so they are interpreted separately from the headline timing campaign.



d	FAISS	cuVS	CAGRA	GGNN
2	3.45×	1.65×	1.48×	0.32×
3	3.02×	1.45×	1.30×	0.28×
4	2.35×	1.13×	1.02×	0.22×
5	3.15×	1.50×	1.38×	0.28×
6	10.71×	4.96×	4.47×	0.50×
7	37.17×	17.52×	15.24×	1.57×
8	41.23×	19.41×	16.88×	1.81×
9	35.68×	17.13×	14.79×	1.59×
10	32.36×	15.55×	13.56×	1.44×
geo.	10.4×	5.0×	4.4×	0.64×

Figure 1: Headline benchmark on detector data at N=5 M, k=40. Left: dimensional scaling of wall-clock time. FastGraph is the fastest among the tested dimension-general exact GPU baselines across the full d=2–10 range. Right: speedups of FastGraph over each backend at the same operating point. FAISS and cuVS BF are exact; CAGRA-nnd and GGNN are approximate baselines. Recall was not measured for these timed configurations; Section 5.7 reports a separate diagnostic experiment.

5.1. Dimensional scaling on detector-derived features

The right-hand table in Figure 1 reports the cross-method speedups of FastGraph at the headline operating point. FastGraph is the fastest among the tested dimension-general *exact* GPU kNN baselines at every dimension we measure and has a 4.4× geomean advantage over the approximate CAGRA-NN-Descent build. The GGNN approximate baseline does win on raw wall-clock at low d. Section 5.7 reports recall separately for the available diagnostic configurations; these do not establish a matched-quality comparison for the headline timings. The relative advantage is widest at d=7–10. Geomean across d=2–10 at this operating point is 10.4× over FAISS, 5.0× over cuVS BF (the strongest exact GPU baseline), and 4.4× over CAGRA-NN-Descent. Section 5.5 separately reports the internal axis-aligned ablation that motivated the PCA-subspace design; the headline table therefore reports only the PCA-subspace method.

FastGraph's runtime drops markedly from d=5 through d=7, while FAISS and cuVS BF remain approximately flat across the measured dimensional range. The wider speedup therefore comes primarily from faster FastGraph execution, rather than increasing brute-force timings. Because each successive d appends a specific stored feature column, this change cannot be attributed to dimensionality or PCA anisotropy alone without feature-subset and normalization controls.

5.2. Dataset-size scaling

Figure 2 reports dataset-size scaling at fixed k=40 for d=3 and d=8 on the common five-point `mps:100` grid. Separation is strongest in the PCA-favorable regime.

Figure 3 checks that the headline dimensional trend is not an artifact of the single k=40 operating point. Across $k \in \{10, 40, 100\}$, the low-dimensional constant-factor regime and the larger $d \geq 6$ speedups remain visible.

5.3. Controlled synthetic benchmarks

On isotropic Gaussian $\mathcal{N}(0, I_d)$ data, FastGraph wins decisively at low dimensions, while dense brute-force GPU matmul (cuVS BF) becomes competitive as d grows. This is consistent with the algorithmic interpretation: the PCA step has no anisotropy to exploit on isotropic

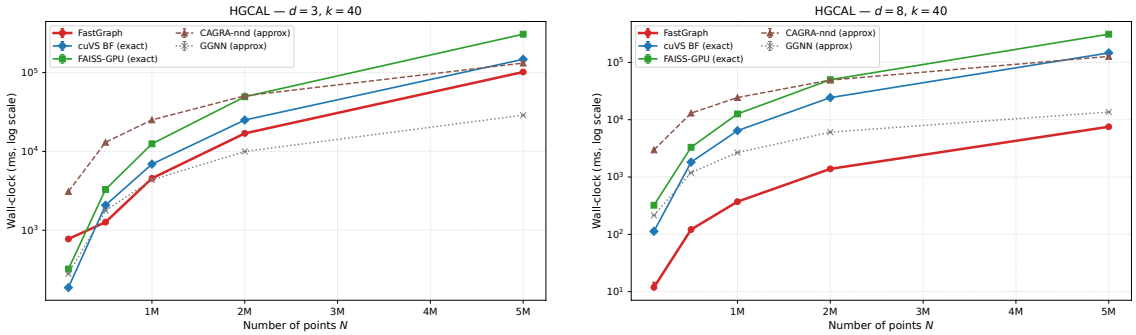


Figure 2: Dataset-size scaling on HGCAL at k=40 for d=3 (left) and d=8 (right). FastGraph maintains a speed advantage over exact baselines most clearly in the higher-dimensional PCA-favorable regime.

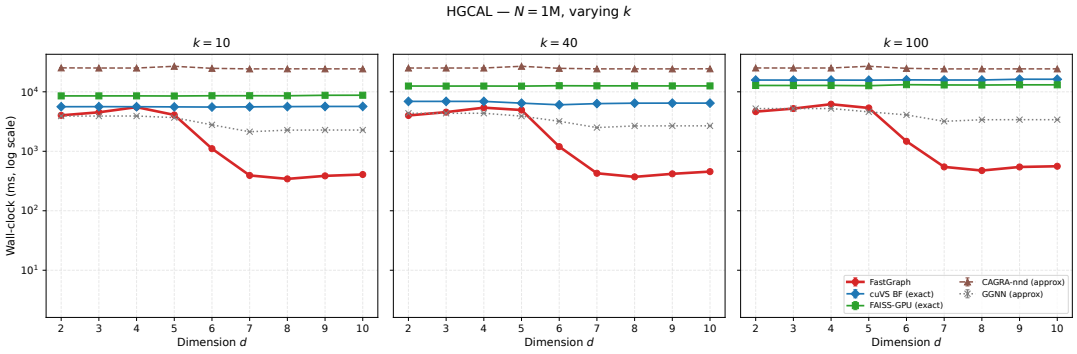


Figure 3: Wall-clock at N=1 M for $k \in \{10, 40, 100\}$ across the full dimensional range. The qualitative pattern of Figure 1 is preserved across k.

data, so the gain comes from ordinary cell-list pruning rather than from a favorable learned basis. At low d, cell-list pruning still dominates the dense $\mathcal{O}(N^2 d)$ matmul of the exact GPU baselines, giving FastGraph a 55× speedup over cuVS BF at d=3 and a 4.8× speedup at d=5 (Table 1). As d grows the number of neighboring cells grows combinatorially, the binning savings collapse, allowing dense matmul on a well-tuned BLAS path to overtake cell-list search. Figure 4 shows the full dimensional sweep and two dataset-size scans at d=3 and d=8.

Table 1: FastGraph speedup over GPU baselines on synthetic Gaussian $\mathcal{N}(0, I_d)$ data at N=10^6, k=40 (median of 3 reps on the fixed-seed synthetic Gaussian benchmark set used throughout this paper). The HGCAL detector-data wins at $d \geq 6$ (Figure 1) are not reproduced on isotropic data, showing that the detector-data result does not follow from dimensionality and cell-list binning alone.

d	vs cuVS BF	vs FAISS-GPU	vs CAGRA-nnd
3	55.2×	108.6×	205.8×
5	4.8×	9.4×	18.0×
7	0.8×	1.7×	3.2×

The implication for the headline claim is narrower: the HGCAL detector wins at d=6–10 (Section 5.1) are consistent with PCA exploiting structure absent from isotropic synthetic data, but the current feature-prefix experiment does not isolate dimensionality, feature

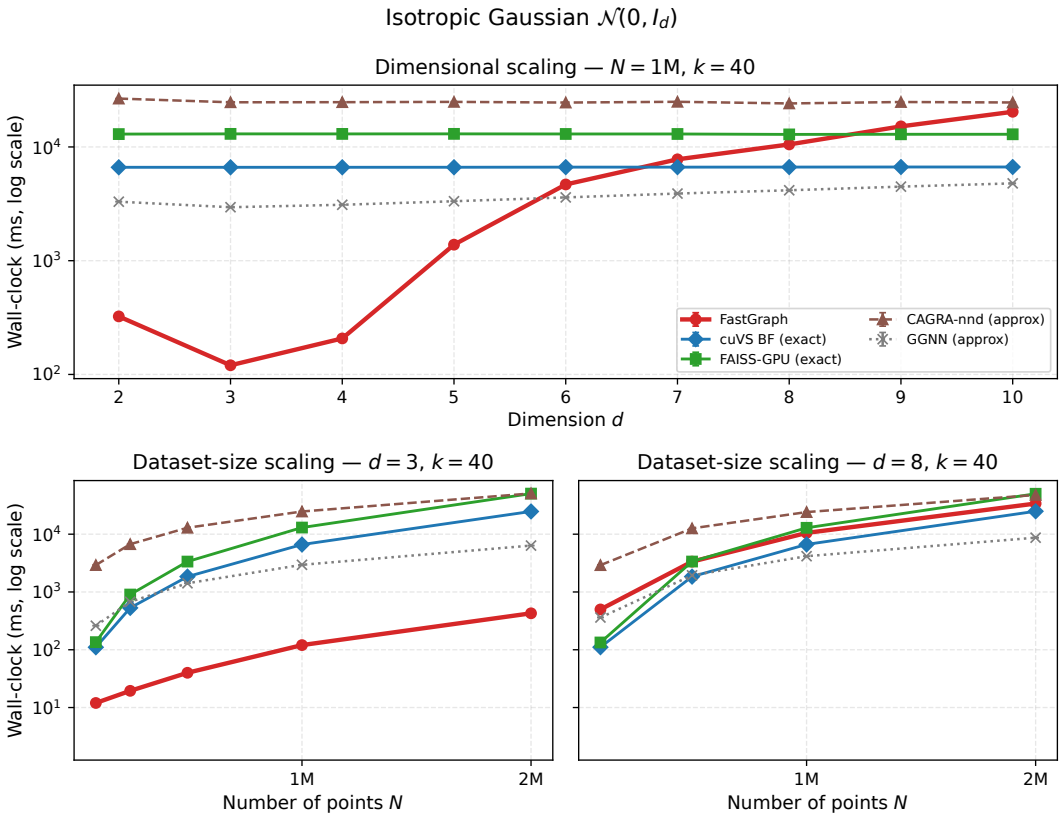


Figure 4: Controlled synthetic Gaussian benchmarks. Top: dimensional scaling at N=1 M, k=40. Bottom: dataset-size scaling at d=3 and d=8, k=40.

identity, units, and anisotropy from one another.

5.4. Memory footprint

The runtime advantage must be considered alongside storage costs. At N=5 M, d=3, k=40, the synchronized post-call delta is about 5.15 GiB for FastGraph versus 1.58–2.82 GiB for the other backends. FastGraph returns int64 indices and float32 distances, whose $N \times k$ tensors alone occupy about 2.24 GiB (2.40 GB) at this point; the remainder includes live tensors and allocator state retained after the call. CAGRA and GGNN return different output representations, so this is an as-used resident-footprint comparison rather than an output-normalized workspace comparison.

These values depend on the returned representation as well as the search method. The plot reports each backend at the configuration used elsewhere in this paper. FAISS-GPU `IndexFlatL2` is exact; cuVS BF is exact. CAGRA-nn-descent and GGNN are approximate, and recall is not measured for these timed configurations. We report the memory each backend uses *as benchmarked*; the output and index representations differ, so this is not an output- or quality-normalized workspace comparison.

The measurement itself also limits the interpretation: memory is reported from synchronized before/after NVML samples. A Python-thread poller was also recorded, but several external backends hold the Python GIL and starve that poller during the call; those transient samples are therefore excluded rather than presented as comparable peak measurements.

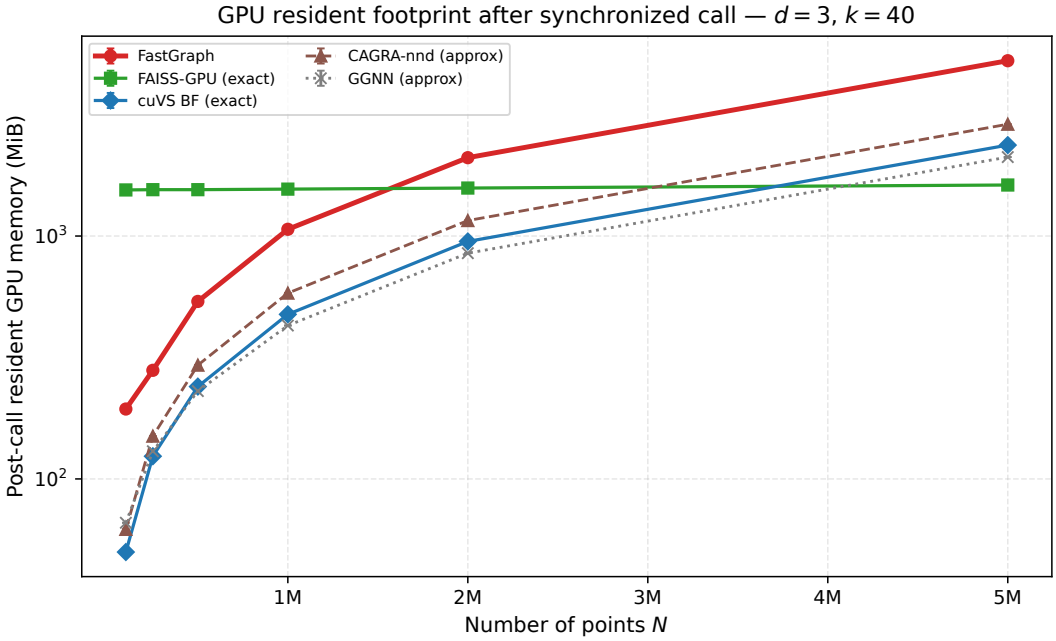


Figure 5: Post-call GPU memory at d=3, k=40. Each value is the change in device-wide NVML used memory between a pre-call baseline and a synchronized post-call sample while the caller still holds the backend's outputs. Values include live outputs plus allocator and index state; they are not transient peak-workspace measurements.

5.5. Binning dimensionality ablation

The binning subspace size k (`max_bin_dims`) is the most important FastGraph hyperparameter. We set the default to k=3 because it is the fastest PCA-subspace setting at the reference cell d=8, N=5 M, k_{nn}=40. Table 2 compares this choice with the axis-aligned cell list over the evaluated kernel surface $k \in \{2, 3, 4, 5\}$, using medians over three repetitions.

Table 2: Effect of the binning subspace size k on FastGraph wall-clock at d=8, N=5 M, k_{nn}=40, median of three repetitions. The axis-aligned variant's optimum within the evaluated kernel surface is k=2; the PCA-subspace variant's runtime optimum, and therefore the default used in the paper, is k=3.

k (bin dims)	axis (s)	PCA (s)	Speedup (axis/PCA time)
2	199.37	11.39	17.5×
3	204.65	**7.48**	**27.4**×
4	353.78	11.58	30.6×
5	242.90	28.10	8.6×

Both variants are non-monotonic in k, but their optima differ: the axis-aligned cell list is fastest at k=2, while the PCA-subspace variant is fastest at k=3. The largest matched-k speedup occurs at k=4, but k=3 is the setting used by FastGraph because it minimizes the PCA-subspace wall-clock. Across the sweep, PCA-subspace binning is 8.6–30.6× faster at matched k, and comparing each variant at its own optimum gives a 26.6× speedup.

We expose k as a user-facing hyperparameter rather than hard-coding it: changing k trades grid size and enumeration cost against the tightness of projected distance bounds, and the same kernel binary supports all four instantiations. The default k=3 is HGCAL-tuned

and should be revalidated when deploying FastGraph on datasets with different intrinsic dimension.

5.6. Three-dimensional baseline: CLOVER head-to-head

CLOVER [19] provides a direct comparison with a 3D-only exact-GPU kNN method. Its released implementation requires $d = 3$; we compare it with FastGraph on the first three stored HGCAL feature columns and on two synthetic distributions. The HGCAL column semantics are unavailable, so these inputs are treated as numerical feature triples rather than identified physical spatial coordinates.

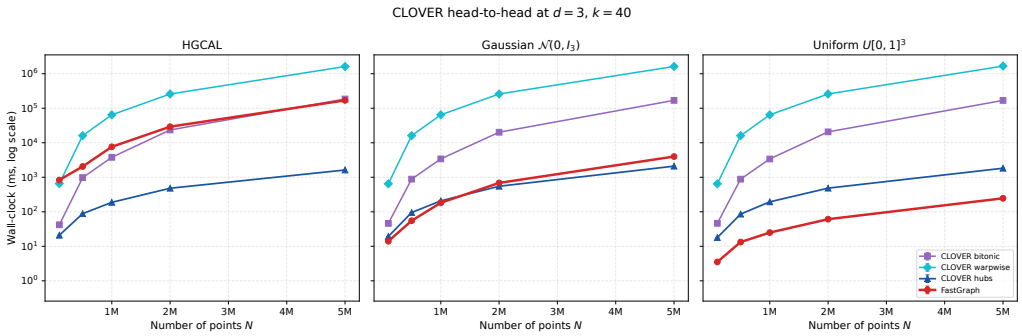


Figure 6: FastGraph vs CLOVER at d=3, k=40, across three data distributions: HGCAL recHits (left), synthetic Gaussian $\mathcal{N}(0, I_3)$ (center), synthetic uniform on $[0, 1]^3$ (right). CLOVER is only implemented for D=3, so d=3 is the only dimensionality at which a direct comparison is possible.

The result splits along the structure of the input distribution. On uniform-cube data, where bin occupancy is comparatively regular, FastGraph is 5–8× faster than CLOVER hubs. On isotropic Gaussian data the two methods are within roughly a factor of two. On anisotropic HGCAL recHits, CLOVER's hubs index is much faster at d=3: with the default k_{bin}=3, FastGraph short-circuits to the ordinary axis-aligned cell-list path, while CLOVER uses a 3D density-adaptive spatio-graph. This is a favorable regime for CLOVER and a limitation of FastGraph's axis-aligned control, which bypasses PCA at d=3.

The deployment regime targeted by FastGraph is the *learned latent space* of a dynamic-graph network. The original GravNet formulation uses a four-dimensional learned space for neighbor construction [3]; this work evaluates that point and nearby low-to-moderate dimensions through d=10. CLOVER, LBVH-kNN, and the RT-core methods considered here do not provide a released $d \geq 4$ implementation for a direct comparison.

5.7. Recall evaluation

We use an archived broad recall sweep against a brute-force reference on HGCAL data up to N=5×10^5. This diagnostic invokes the axis-aligned `binned_select_knn` control rather than the PCA wrapper. Its CAGRA rows use the default graph build and `itopk_size`=512; its GGNN rows use $\tau_{\text{query}} = 0.8$. This archived experiment is separate from the headline timing campaign, which uses CAGRA–NN-Descent. We use these archived outputs only to report recall; none of their timing values enters the performance comparisons.

Because multiple recHits can have tied distances in the selected feature space, neighbor indices are not unique. We use *distance-based recall*, counting a predicted neighbor as correct when its squared distance $\widehat{D}_{v,j}$ satisfies

$$\widehat{D}_{v,j} \leq (1 + 10^{-4}) D_{v,(K)} + 10^{-10},$$

where $D_{v,(K)}$ is the K-th brute-force squared distance, including self. Recall is the fraction of the NK returned entries satisfying this threshold. FastGraph and its reference use element-wise squared ℓ_2 distances, which avoid the cancellation that can affect coincident-hit cases under the BLAS expansion. FAISS-GPU `IndexFlatL2` is exact under its own L2 kernel and is shown as such. Exactness here means satisfying the fixed distance criterion, not producing a unique graph topology when ties are present.

At the common representative cell, the FastGraph axis control attains recall = 1.0, while neither approximate backend reaches exact recall at its recorded highest-quality setting. Table 3 reports means over the three repetitions of that cell. We do not report a cross-grid mean because the archived default-build CAGRA data cover $d = 2$–7, whereas the GGNN and FastGraph data extend through $d = 10$.

Table 3: Distance-based recall at d=3, N=5×10^5, K=40. CAGRA uses its default graph build with `itopk_size`=512; GGNN uses $\tau_{\text{query}} = 0.8$. Values are means over three repetitions.

Method	distance-based recall
FastGraph axis control (exact)	1.000
FAISS-GPU `IndexFlatL2` (exact)	1.000
CAGRA, default build (approximate)	0.818
GGNN (approximate)	0.410

These values establish the quality gap only in the recorded diagnostic configurations. They do not measure recall for the headline timing campaign.

The PCA path is validated separately with the released `binned_select_knn_pca` entry point. Dedicated GPU tests compare the complete sorted distance multiset returned for every query with an element-wise FP32 brute-force reference on seeded Gaussian inputs at $d \in \{4, 5, 8, 10\}$, $K \in \{16, 40, 128\}$, and N up to 8,000. They also exercise random PCA subsampling and a three-segment `row_splits` input. All tested indices are valid, unique, and segment-local; returned distances match recomputed distances, and every distance multiset matches brute force within the test tolerances. Returned versus recomputed squared distances use `atol`=10^{-6} and `rtol`=10^{-5}; sorted distance multisets versus brute force use `atol`=10^{-6} and `rtol`=10^{-3}. Each comparison requires $|a - b| \leq \texttt{atol} + \texttt{rtol}|b|$, with b the reference value. These finite tests validate the exercised configurations, not all FP32 inputs.

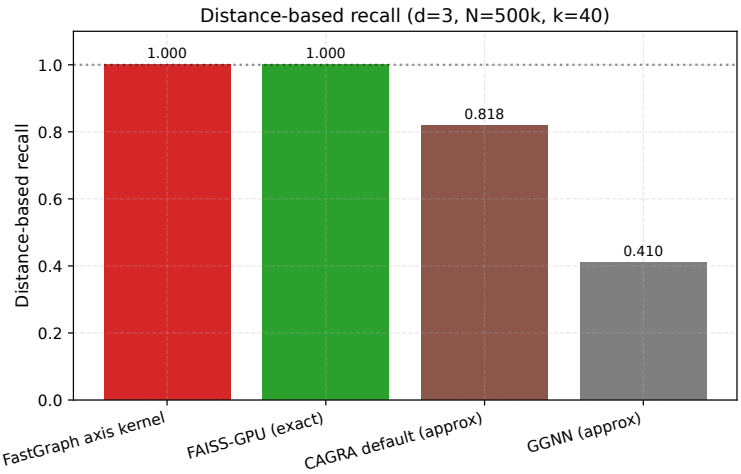


Figure 7: Distance-based recall at the representative HGCAL diagnostic cell d=3, N=5 × 10^5, K=40. FastGraph and FAISS-GPU satisfy their exact distance conventions; the approximate results use default-build CAGRA with `itopk_size`=512 and GGNN with $\tau_{\text{query}} = 0.8$. This figure reports recall only.

5.8. Connection to calorimeter reconstruction

GravNet-style layers [3] construct a fresh kNN graph in a learned latent space at every forward pass, so the kNN substrate is on the gradient path of every training step. FastGraph provides an implementation path into this setting through `GravNetOp`, a PyTorch module that wraps coordinate transformation, kNN graph construction (via the opt-in `use_pca=True` PCA-subspace path of Section 3), and message passing into a single autograd-compatible operation. Gradients flow through the selected distances and the input coordinates; the discrete neighbor indices and the per-call projection V_k are held fixed across each forward–backward pair so that graph selection is consistent within the pair; atomic gradient accumulation is not guaranteed to be bitwise deterministic.

Within the reconstruction chain described in the Introduction, this operation supplies neighborhoods for hit-feature aggregation. The later assignment of hits to showers and the inference of particle properties remain tasks of the trained network and its reconstruction procedure. FastGraph changes how the specified neighborhoods are found, while retaining full-coordinate distance evaluation; it does not define a new shower-clustering objective. In particular, the million-point timing inputs are not individual collision events, and the separate segmented correctness tests are not event-batched performance measurements. The present evidence supports the cost of the search component on the reported inputs and its software integration, without demonstrating a change in shower separation, energy response, or event throughput.

6. Limitations

The detector-feature results establish a useful operating regime, but their interpretation depends on both the indexing geometry and the relationship of the benchmark to reconstruction. Both FastGraph and its axis-aligned ablation bound the grid at n_{bins}^k for a small binning dimension k, independent of input dimension d. FastGraph uses a data-adaptive orthonormal basis rather than tying the grid to a fixed set of input columns. In exact arithmetic this avoids privileging the first input columns as binning axes; the sampled finite-precision fit

need not be bitwise invariant under column permutations. PCA is not invariant to rescaling individual features: their units and preprocessing change the covariance and therefore the selected subspace. The method still requires a small k to keep the grid tractable.

For a given metric, pruning is most useful when projected separations provide informative bounds on full-space distances. Its effectiveness therefore depends on how much variation the leading components capture. On the detector-feature matrix used here, FastGraph's PCA-subspace binning is substantially faster than axis-aligned cell lists (Section 5.5, geomean speedups in Section 5.1). On isotropic data, the leading subspace is an arbitrary orthonormal k-dimensional projection and FastGraph has no preferred principal directions to exploit; in that regime FastGraph outperforms dense GPU brute force only while the cell-list pruning advantage outweighs the constant factor of an optimized BLAS `gemm`. Section 5.3 documents this on Gaussian $\mathcal{N}(0, I_d)$ inputs and is the reason the timing claim is restricted to the measured detector-feature configurations rather than generalized to all HGCAL workloads. Whether trained learned-latent embeddings exhibit similarly favorable spectra remains to be tested.

Setup costs also affect transfer to realistic events. PCA estimation adds a sampled low-rank fit and an $O(Ndk)$ projection, both included in the measured FastGraph timings. Their relative cost can increase for smaller segments; the crossover at which an axis-aligned or brute-force search becomes preferable requires measurement on those workloads. The current implementation targets a single GPU; multi-GPU or distributed kNN graph construction is not supported. The discrete neighbor indices are not differentiable; FastGraph's autograd contract is over the continuous distances and coordinates only, matching every other hard-selection kNN layer. Approximate methods can be faster in some cells: GGNN wins on raw wall-clock for d=2–6, whereas FastGraph is faster than GGNN for d=7–10 and faster than CAGRA throughout the headline sweep. Recall at these timed operating points remains unmeasured. The appropriate backend therefore depends on dimension, required graph quality, and end-to-end cost.

The detector-derived timing study is a kernel stress test rather than a deployment benchmark: it concatenates recHits and passes each slice as a single segment. Event-preserving row splits can change both the problem size and the spatial density seen by a cell list. Likewise, the benchmark features are detector-derived inputs, not embeddings sampled from a trained model. Event-batched timings and learned-latent coordinates are therefore required before extrapolating the reported speedups to end-to-end training throughput. We defer those model-level measurements to future work and make no end-to-end throughput claim here. In particular, reducing search time alone does not establish improved energy resolution, particle-identification performance, or shower separation. Extending the evaluated input dimension beyond d=10 is secondary to those two validation steps; higher-dimensional feature-space graph layers remain outside the present claim. A projected-space LBVH is another possible extension when uniform bins cease to prune effectively.

Choosing between exact and approximate neighborhoods consequently requires validation in the reconstruction model. General graph-learning studies show that changes in connectivity can affect predictions [32, 33], but they do not establish the physics impact of approximate neighbor search in HGCAL reconstruction. Exact search avoids a recall-tuning parameter; it does not by itself guarantee a unique graph under ties, bitwise reproducible gradients, or controlled physics systematics. Moreover, the exact-arithmetic pruning argument does not bound FP32 roundoff, and the PCA-path correctness tests stop at $N = 8{,}000$. Large-scale detector-feature boundary cases and matched-recall timing curves remain unmeasured.

Backend compatibility is another restriction on the comparison. The default CAGRA IVF-PQ build path fails in the 5 M-point timing campaign at $d \geq 5$ (Section 4); the NN-Descent fallback succeeds across the full dimensional sweep but is 16.88× slower than FastGraph at the d=8 reference cell. We use the NN-Descent numbers for the headline timing campaign; the archived recall diagnostic is explicitly labeled as default-build CAGRA. This is an observed compatibility issue for the tested dataset and software configuration, not evidence of a generic CAGRA limitation or a diagnosis of the internal failure cause.

7. Data and Code Availability Statement

The FastGraph implementation described in this paper is available at `https://github.com/AgarwalAarush/FastGraphCompute`, as a fork of the upstream `FastGraphCompute` library. The immutable `pca-fgc-paper-v1.3.4` release tags pin the exact FastGraph, benchmark-harness, plotting, and CLOVER-fork revisions used for the reported experiments; the release is built against the CUDA 12.1 toolchain and reports FastGraphCompute package version `1.2.0`. The evaluated and recommended kernel surface covers binning subspace size $k \in \{2, 3, 4, 5\}$; the tag also contains experimental $k = 6, 7$ dispatches, which are excluded from the headline configuration. The benchmarking harness and runner scripts used to produce the figures live in a companion repository, `https://github.com/AgarwalAarush/fgc-performance`. The CLOVER head-to-head used a fork of `https://github.com/ampslab/clover-knn` at `https://github.com/AgarwalAarush/clover-knn` with local source modifications (`k_values` aligned to the paper's k=40, the hardcoded synthetic-N template chain in `main()` disabled in favor of the mesh path, and the build retargeted to `sm_80` for A100). The revised manuscript source is maintained at `https://github.com/AgarwalAarush/fastgraph-paper`.

The detector-derived stress tests use CMS HGCAL simulation data (recHit feature vectors) accessed via an internal collaboration dataset of 1.16 billion calorimeter hits with up to ten stored numerical features per hit; their names and units are not included in the available artifact. The benchmark concatenates events and does not release event boundaries or learned model embeddings. The data itself is internal to the CMS collaboration; synthetic Gaussian and uniform datasets used in Sections 5.3 and 5.6 are generated on the fly from fixed random seeds and are fully reproducible from the released code. CMS collaboration members can reproduce the HGCAL results directly; external parties should contact the corresponding author.

8. Conclusion

Separating overlapping showers in a high-granularity calorimeter motivates graph-based reconstruction methods that repeatedly find neighbors in learned coordinate spaces. FastGraph addresses this computational step with a PCA-subspace cell list and full-coordinate distance evaluation. The contractive projection supplies an exact-arithmetic search criterion, while the released CUDA/PyTorch implementation provides distance gradients and an opt-in GravNet path.

On the detector-derived recHit stress test at single-segment N=5 M, k=40, FastGraph is faster than the tested dimension-general exact GPU baselines across d=2–10. At d=8, construction takes 7.5 s, with speedups up to 41× over FAISS-GPU and 19× over cuVS brute force. The 17× advantage over approximate CAGRA-NN-Descent is a timing comparison without matched recall. Comparing PCA and axis-aligned binning at their respective optima

gives a 26.6× ratio at this reference cell. These gains depend on the input distribution: isotropic synthetic inputs become unfavorable as dimension grows, and CLOVER is about 100× faster at the largest d=3 HGCAL comparison point, where FastGraph bypasses PCA.

The demonstrated advance is reduced search cost on the stated detector-feature workloads. Their unknown feature semantics and single-segment construction prevent a direct inference about event-local learned-coordinate graphs. Measuring that workload, validating its numerical behavior at scale, and profiling complete reconstruction models are the next steps toward establishing the method's physics benefit. No improvement in reconstruction accuracy or end-to-end throughput is claimed by the present measurements.

Declaration of Generative AI and AI-Assisted Technologies in the Manuscript Preparation Process

During the preparation of this work, the authors used OpenAI Codex for language editing, software review, and reproducibility checks. The authors reviewed and edited the output as needed and take full responsibility for the content of the published article.

References

[1] F. Pitters, The CMS high-granularity calorimeter for operation at the high-luminosity LHC, on behalf of the CMS Collaboration (2018). arXiv:1802.05987, doi:10.48550/arXiv.1802.05987.
URL `https://arxiv.org/abs/1802.05987`

[2] S. R. Qasim, K. Long, J. Kieseler, M. Pierini, R. Nawaz, Multi-particle reconstruction in the high granularity calorimeter using object condensation and graph neural networks, in: EPJ Web of Conferences, Vol. 251, 2021, p. 03072. doi:10.1051/epjconf/202125103072.

[3] S. R. Qasim, J. Kieseler, Y. Iiyama, M. Pierini, Learning representations of irregular particle-detector geometry with distance-weighted graph networks, The European Physical Journal C 79 (7) (2019) 608.

[4] J. Kieseler, Object condensation: One-stage grid-free multi-object reconstruction in physics detectors, graphs, and images, The European Physical Journal C 80 (9) (2020) 886.

[5] S. Bhattacharya, et al., GNN-based end-to-end reconstruction in the CMS phase-2 high-granularity calorimeter, arXiv preprint arXiv:2203.01189 (2022).

[6] L. Verlet, Computer "experiments" on classical fluids. i. thermodynamical properties of Lennard-Jones molecules, Physical Review 159 (1) (1967) 98–103. doi:10.1103/PhysRev.159.98.

[7] M. P. Howard, J. A. Anderson, A. Nikoubashman, S. C. Glotzer, A. Z. Panagiotopoulos, Efficient neighbor list calculation for molecular simulation of colloidal systems using graphics processing units, Computer Physics Communications 203 (2016) 45–52. doi:10.1016/j.cpc.2016.02.003.

[8] J. Johnson, M. Douze, H. Jégou, Billion-scale similarity search with GPUs, IEEE Transactions on Big Data 7 (3) (2021) 535–547.

[9] J. Feydy, J. Glaunès, B. Charlier, M. Bronstein, Fast geometric learning with symbolic matrices, in: Advances in Neural Information Processing Systems, Vol. 33, 2020, pp. 14448–14462.

[10] A. Dashti, I. Komarov, R. M. D'Souza, Efficient computation of k-nearest neighbour graphs for large high-dimensional data sets on GPU clusters, PLOS ONE 8 (9) (2013) e74113.

[11] I. Komarov, A. Dashti, R. M. D'Souza, Fast k-NNG construction with GPU-based quick multi-select, PLOS ONE 9 (5) (2014) e92409.

[12] M. Fey, J. E. Lenssen, Fast graph representation learning with PyTorch Geometric, arXiv preprint arXiv:1903.02428 (2019). doi:10.48550/arXiv.1903.02428.

[13] S. R. Qasim, Multi-particle reconstruction with dynamic graph neural networks, Ph.D. thesis, Manchester Metropolitan University, see §8.2.2 "Binned K Nearest Neighbours (KNN)", Algorithm 3, pp. 154–158 (2023).
URL `https://e-space.mmu.ac.uk/632184/`

[14] R. F. Sproull, Refinements to nearest-neighbor searching in k-dimensional trees, Algorithmica 6 (1991) 579–589. doi:10.1007/BF01759061.

[15] Y. Zhu, RTNN: Accelerating neighbor search using hardware ray tracing, in: Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP), ACM, 2022, pp. 76–89. doi:10.1145/3503221.3508409.

[16] V. Nagarajan, D. Mandarapu, M. Kulkarni, RT-kNNS unbound: Using RT cores to accelerate unrestricted neighbor search, in: Proceedings of the 37th ACM International Conference on Supercomputing (ICS), ACM, 2023, pp. 289–300. doi:10.1145/3577193.3593738.

[17] D. K. Mandarapu, V. Nagarajan, A. Pelenitsyn, M. Kulkarni, Arkade: k-nearest neighbor search with non-Euclidean distances using GPU ray tracing, in: Proceedings of the 38th ACM International Conference on Supercomputing (ICS), ACM, 2024, pp. 14–25. doi:10.1145/3650200.3656601.

[18] J. Jakob, M. Guthe, Optimizing LBVH-construction and hierarchy-traversal to accelerate kNN queries on point clouds using the GPU, Computer Graphics Forum 40 (1) (2021) 124–137.

[19] V. Kamel, H. Yan, S. Chester, CLOVER: A GPU-native, spatio-graph-based approach to exact kNN, in: Proceedings of the 39th ACM International Conference on Supercomputing (ICS), ACM, 2025, pp. 236–249. doi:10.1145/3721145.3730415.

[20] W. Zhao, S. Zhang, Y. Ding, Z. Tan, J. Liu, D. Zhan, SONG: Approximate nearest neighbor search on GPU, in: 2020 IEEE 36th International Conference on Data Engineering (ICDE), IEEE, 2020, pp. 1033–1044.

[21] F. Groh, L. Ruppert, P. Wieschollek, H. P. A. Lensch, GGNN: Graph-based GPU nearest neighbor search, IEEE Transactions on Big Data 9 (1) (2023) 267–279.

[22] H. Ootomo, A. Naruse, C. Nolet, R. Wang, T. Feher, Y. Wang, CAGRA: Highly parallel graph construction and approximate nearest neighbor search for GPUs, in: 2023 IEEE International Conference on Big Data (BigData), IEEE, 2023, pp. 33–42.

[23] H. Wang, W.-L. Zhao, X. Zeng, Large-scale approximate k-NN graph construction on GPU, arXiv preprint arXiv:2103.15386 (2021).

[24] Y. A. Malkov, D. A. Yashunin, Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs, IEEE Transactions on Pattern Analysis and Machine Intelligence 42 (4) (2020) 824–836.

[25] R. Guo, P. Sun, E. Lindgren, Q. Geng, D. Simcha, F. Chern, S. Kumar, Accelerating large-scale inference with anisotropic vector quantization, in: Proceedings of the 37th International Conference on Machine Learning (ICML), 2020, pp. 3887–3896.

[26] E. Bernhardsson, Annoy: Approximate nearest neighbors in C++/Python optimized

for memory usage and loading/saving to disk, Open-source software library, Spotify, version 1.17.3 (2013).
URL https://github.com/spotify/annoy

[27] Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, J. M. Solomon, Dynamic graph CNN for learning on point clouds, ACM Transactions on Graphics 38 (5) (2019) 1–12.

[28] H. Qu, L. Gouskos, ParticleNet: Jet tagging via particle clouds, Physical Review D 101 (5) (2020) 056019.

[29] X. Ju, et al., Performance of a geometric deep learning pipeline for HL-LHC particle tracking, The European Physical Journal C 81 (10) (2021) 876.

[30] A. Lazar, et al., Accelerating the exa.TrkX pipeline, Computing and Software for Big Science 6 (1) (2022) 1–9.

[31] N. Choma, et al., Track seeding and labelling with embedded-space graph neural networks, arXiv preprint arXiv:2007.00149 (2020).

[32] D. Zügner, A. Akbarnejad, S. Günnemann, Adversarial attacks on neural networks for graph data, in: Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD), ACM, 2018, pp. 2847–2856. doi:10.1145/3219819.3220078.

[33] J. Gasteiger, S. Weißenberger, S. Günnemann, Diffusion improves graph learning, in: Advances in Neural Information Processing Systems 32 (NeurIPS), 2019, pp. 13333–13345, arXiv:1911.05485.